SENDA-TR-2026-03 · TECHNICAL REPORT

Display Determinism Across JavaScript Runtimes for Adaptive ODE Solvers

The SEIR mass-defect case: raw floats differ, the published bound does not

Abraham Haddioui · 2026-11-07 · version 1.0

Technical report · not peer reviewed · MIT

Abstract

Adaptive ODE solvers are sensitive to floating-point details: a tiny change in an embedded error estimate can alter a step-size decision and the entire trajectory. This report studies the SEIR integrator in the SENDA precision lab, which uses the Dormand–Prince 5(4) pair with per-step error control. Running the same module in Node.js 22.22.2 and Google Chrome 156.0.0.0 (headless) over four parameter settings produced different raw mass-conservation errors—between 1.02×10^{-12} and 2.50×10^{-12}—while the accepted step counts agreed in all four cases and the published display `< 1e-9` was identical. We formalize why: an upper-bound display is deterministic when every runtime's raw value lies below the bound, even if the raw values differ. A fixed-point display would not have this property; at twelve decimal places the default case would render `0.000000000002` in Node and `0.000000000001` in Chrome. The result is a design rule for numerical claims, not a claim that adaptive solvers are bit-reproducible.

Keywords: adaptive Runge–Kutta, SEIR, floating-point, cross-runtime determinism, display contract, reproducibility.

1. Introduction

A dashboard that reports "mass error below 10^{-9}" is making a different statement from one that reports "mass error 1.82×10^{-12}." The first is a bound that can survive implementation details; the second is a raw float that may not.

This report examines that distinction in a concrete solver. The SENDA precision lab integrates a four-compartment SEIR model with the Dormand–Prince 5(4) embedded pair [1]. The same JavaScript module runs in the browser and in Node.js. We measured the raw maximum mass defect and the rendered text in both engines under four parameter settings.

The contributions are:

1. a precise description of the integrator and its error controller;
2. a cross-runtime matrix of raw values, step counts, and displays;
3. a proof that upper-bound displays are deterministic under a simple condition;
4. a counterexample showing that fixed-point displays are not;
5. a verification recipe using PCI v0.1.

Section 2 describes the model and integrator. Section 3 analyzes divergence. Section 4 proves the display result. Section 5 reports the experiment. Section 6 connects the result to PCI. Section 7 relates prior work. Section 8 lists limitations.

---

2. Model and integrator

2.1 SEIR equations

Let S, E, I, R be susceptible, exposed, infectious, and recovered counts, with $N = S + E + I + R$. The model is

$$\begin{aligned} \dot{S} &= -\frac{\beta c}{N} S I, \\ \dot{E} &= \frac{\beta c}{N} S I - \sigma E, \\ \dot{I} &= \sigma E - \gamma I, \\ \dot{R} &= \gamma I, \end{aligned} \tag{1}$$

where c is contacts per day, β is the per-contact transmission probability, $\sigma = 1/\mathrm{latencyDays}$, and $\gamma = 1/\mathrm{infectiousDays}$. The basic reproduction number is

$$R_0 = \frac{\beta c}{\gamma} = \beta c \, \mathrm{infectiousDays}. \tag{2}$$

The exact flow preserves N: summing (1) gives $\frac{d}{dt}(S + E + I + R) = 0$.

2.2 Dormand–Prince 5(4)

The implementation uses the seven-stage Dormand–Prince pair. Each step produces a fifth-order candidate y_5 and a fourth-order candidate y_4. The local error estimate is

$$e_i = \frac{|y_{5,i} - y_{4,i}|}{\mathrm{atol} + \mathrm{rtol}\,\max(|y_i|, |y_{5,i}|)}, \qquad e = \max_i e_i. \tag{3}$$

The defaults are $\mathrm{rtol} = 10^{-7}$, $\mathrm{atol} = 10^{-9}$, and $\Delta t_{\max} = 0.5$ days. A step is accepted when $e \le 1$. On acceptance the next step is

$$h' = \min\left(\Delta t_{\max},\; h \cdot \mathrm{clamp}\left(0.9\,e^{-1/5},\, 0.2,\, 5\right)\right), \tag{4}$$

and on rejection

$$h' = \max\left(h \cdot \max\left(0.2,\, 0.9\,e^{-1/5}\right),\; 10^{-9}\right). \tag{5}$$

The implementation samples at integer days and applies a non-negativity projection $y_i \leftarrow \max(0, y_i)$ at sample points. The mass defect is measured after the projection:

$$e_{\mathrm{mass}}(t) = \left|S(t) + E(t) + I(t) + R(t) - N\right|, \qquad e_{\max} = \max_{t \in \{1,\dots,90\}} e_{\mathrm{mass}}(t). \tag{6}$$

Two details matter for reproducibility. First, the error estimate (3) is a maximum over four scaled differences, so a tiny change in one component can change the accept/reject decision. Second, the controller (4)–(5) is discontinuous at the accept boundary and clamped at both ends; small perturbations can be amplified or absorbed.

---

3. Sources of cross-runtime divergence

JavaScript specifies IEEE-754 double arithmetic for the basic operators, but it does not require bit-identical results for every library function or for every optimization. In this solver the plausible sources are:

1. **`Math.pow` in the controller.** The expression `e ** -0.2` is implementation-approximated. A one-ulp difference changes h' and can shift subsequent samples.
2. **Fused operations and optimization.** Engines may combine or reorder floating-point operations in ways that preserve the language semantics but change rounding.
3. **Adaptive amplification.** A different step size changes the sequence of stages and the accumulated round-off, even when the continuous solution is the same.
4. **Projection and sampling.** The projection at sample points and the comparison `|t-nextSample| < 1e-9` are additional decision points.

None of these is a bug. They are the normal behavior of floating-point software. The engineering question is what the software publishes.

---

4. Display determinism

Definition 1. Let v_ρ be the raw value produced by runtime ρ and let δ be a display rule. The display is deterministic over a set R of runtimes if $\delta(v_\rho)$ is the same string for every $\rho \in R$.

Lemma 1 (upper bound). Let δ render `< B` when $0 \le v < B$ and `≤ B` when $v = B$, for a constant $B > 0$. If $0 \le v_\rho < B$ for every $\rho \in R$, then δ is deterministic and renders `< B`.

Proof. The output depends only on the predicate $v < B$ and the constant B. Every runtime satisfies the predicate, so the output is the same. ■

Corollary 1. If a claim publishes `upper-bound` with $B = 10^{-9}$ and every observed raw mass error is below 10^{-9}, the rendered text is `< 1e-9` on every observed runtime, regardless of the raw differences.

The corollary is conditional on the bound being correct. A bound that is too small is deterministically wrong; the lemma does not validate the physics.

For fixed-point displays, a margin condition is sufficient.

Lemma 2. Let δ be `fixed:N`. If there is an integer k such that both v and v' lie strictly inside $\big((k - \tfrac{1}{2})10^{-N}, (k + \tfrac{1}{2})10^{-N}\big)$, then $\mathrm{toFixed}(v, N) = \mathrm{toFixed}(v', N)$.

Proof. `toFixed(N)` selects the nearest multiple of 10^{-N} with ties resolved by the specified algorithm. The open interval excludes both tie boundaries, so both values select the same multiple. ■

In the default case, the raw values are approximately 1.819×10^{-12} and 1.023×10^{-12}. At $N = 12$,

```
Node:      0.000000000002
Chrome:  0.000000000001
```

so the fixed-point display is not deterministic. This is not a contrived example; it is the same quantity that the upper-bound display renders identically.

---

5. Experiment

5.1 Protocol

The same module, `lib/precision-lab.mjs`, was loaded in two engines:

- Node.js 22.22.2, importing the module directly;
- Google Chrome 156.0.0.0 in headless mode, loading the module from a local HTTP server.

For each parameter setting, the module was called with population $N = 1000$, latency 3 days, 90 simulated days, and the default tolerances. The raw `maxConservationError`, accepted steps, peak day, and final attack rate were recorded. The display rule was applied independently of the solver.

5.2 Settings and raw results

SETTING	CONTACTS/DAY	B	INFECTIOUS DAYS	NODE MAX ERROR	CHROME MAX ERROR	STEPS N/C
default	8	0.15	5	1.819×10^{-12}	1.023×10^{-12}	233 / 233
dense-high	16	0.50	10	2.501×10^{-12}	1.251×10^{-12}	338 / 338
sparse-low	2	0.05	2	1.023×10^{-12}	1.364×10^{-12}	209 / 209
mid	12	0.30	7	1.023×10^{-12}	1.705×10^{-12}	325 / 325

The raw errors differ in every row. The largest observed value is 2.501×10^{-12}, still more than two orders of magnitude below 10^{-9}. The accepted step counts agree in this matrix; that is an observation, not a theorem.

The final attack rate for `sparse-low` also differed in the last digits: 0.0012496485891168732 in Node and 0.0012496485891165322 in Chrome. The percent display, `0.12%`, agreed.

5.3 Display results

SETTING	NODE TEXT	CHROME TEXT	SAME?
default	`< 1e-9`	`< 1e-9`	yes
dense-high	`< 1e-9`	`< 1e-9`	yes
sparse-low	`< 1e-9`	`< 1e-9`	yes
mid	`< 1e-9`	`< 1e-9`	yes

By Lemma 1, this agreement is expected once both raw values are below the bound. It does not depend on the exact floating-point trajectory.

5.4 Threats to validity

- One machine, two engines, one module version, four settings.
- The browser and Node runs used the same source; a different bundler or minifier could change the trajectory further.
- The experiment measures the module directly, not the rendered React page. The page uses the same function and the same display rule.
- The result is about a display contract, not about the scientific validity of the SEIR parameters.

---

6. PCI claim

The SEIR result can be published as a PCI claim with an `upper-bound` display:

```
{
  "result": {
    "value": 1.8189894035458565e-12,
    "display": { "text": "< 1e-9", "rule": "upper-bound", "bound": 1e-9 }
  },
  "method": {
    "name": "Dormand-Prince RK45",
    "exact": false,
    "tolerances": { "relative": 1e-7, "absolute": 1e-9 }
  },
  "provenance": { "class": "SIMULATED" }
}
```


The verifier checks that the text is exactly `< 1e-9` and that the raw value is strictly below the bound. The Chrome raw value is not in the claim, but it is also below the bound; a second claim can record it, or a claim set can record both. What matters is that the published text is a contract that both runtimes satisfy.

If the producer instead published `fixed:12`, the same claim would fail on one of the runtimes. The verifier would catch the text mismatch only for the claim that was actually sealed; it cannot know about a runtime that was never recorded. This is why the cross-runtime matrix belongs in the evidence package.

---

7. Related work

Floating-point reproducibility is a mature area [2, 3, 4]. The novelty here is not a new summation algorithm; it is the separation between the raw value, the display rule, and the verification contract. Adaptive Runge–Kutta methods and their error control are standard [1, 5]. Reproducible-research packaging [6] and provenance standards [7] provide the surrounding metadata layer. PCI adds a small, offline numerical claim format; SENDA-TR-2026-01 gives the full specification.

---

8. Limitations

- The experiment covers two engines and one solver. It does not establish bit-reproducibility or universal display determinism.
- The upper-bound lemma assumes the bound is correct and the raw value is non-negative. A wrong bound yields a stable but false display.
- The fixed-point lemma is sufficient, not necessary; a particular pair of values may still round to the same text outside the open interval.
- The SEIR parameters are exercise values, not field estimates.

- The mass defect is measured at daily samples after a projection; it is not a continuous-time error bound.
- The paper is a technical report, not a peer-reviewed publication.

---

9. Conclusion

Adaptive solvers can produce different raw floats on different engines, and that is acceptable if the software publishes the right contract. An upper-bound display turns a fragile float into a stable statement; a fixed-point display turns a stable model into a fragile statement. PCI makes that choice explicit and checkable offline.

References

[1] J. R. Dormand and P. J. Prince. “A family of embedded Runge–Kutta formulae.” Journal of Computational and Applied Mathematics, 6(1):19–26, 1980.

[2] D. Goldberg. “What every computer scientist should know about floating-point arithmetic.” ACM Computing Surveys, 23(1):5–48, 1991.

[3] N. J. Higham. Accuracy and Stability of Numerical Algorithms. 2nd ed., SIAM, 2002.

[4] IEEE. “IEEE Standard for Floating-Point Arithmetic.” IEEE 754-2019, 2019.

[5] E. Hairer, S. P. Nørsett, and G. Wanner. Solving Ordinary Differential Equations I: Nonstiff Problems. 2nd ed., Springer, 1993.

[6] S. Soiland-Reyes et al. “Packaging research artefacts with RO-Crate.” Data Science, 5(2), 2022.

[7] W3C. “PROV-Overview: An Overview of the PROV Family of Documents.” W3C Working Group Note, 2013.

Appendix A — Reproduction

```
node scripts/research/cross-runtime.mjs
```

The script computes the Node matrix, launches a local headless Chrome page that imports the same module, compares the raw values and displays, and writes `public/research/data/tr-2026-03-cross-runtime.json`. It performs no network access beyond the loopback server. Set `SENDA_CHROME` to override the browser path.